

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

Тема: «Автоматическая генерация тестов на основе формального описания»

Платформа автоматизации: GitHub + GitHub Actions

Язык реализации тестируемого модуля: C#

Фреймворк тестирования: NUnit

1. Цель и задачи лабораторной работы

Цель: сформировать навыки трансляции функциональных и нефункциональных требований в машинно-читаемую формальную спецификацию, разработки генератора модульных тестов и интеграции процесса в CI/CD-пайплайн на базе GitHub.

Задачи:

1. Изучить принципы контрактного проектирования и автоматизации тест-дизайна.
 2. Преобразовать текстовые требования модуля в формальное описание (YAML) с явной трассировкой требований.
 3. Разработать Python-скрипт, генерирующий параметризованные NUnit-тесты на основе спецификации.
 4. Настроить GitHub Actions для автоматической генерации, компиляции и запуска тестов при каждом push/pull_request.
 5. Подготовить аналитический отчёт с результатами работы пайплайна и оценкой покрытия.
-

2. Теоретические сведения

2.1. Формальная спецификация и контрактное проектирование

Формальная спецификация — строгое, однозначное описание поведения программного модуля в декларативном формате (YAML, JSON, TLA+, OpenAPI). В отличие от текстовых ТЗ, спецификация машиночитаема, верифицируема и может служить источником истины для автоматической генерации тестов.

Контрактное проектирование (Design by Contract) базируется на трёх компонентах:

- **Предусловие (Precondition)** — логическое условие, гарантирующее корректность входных данных. Нарушение предусловия означает ошибку вызывающей стороны.
- **Постусловие (Postcondition)** — условие, описывающее ожидаемое состояние системы или возвращаемое значение после успешного выполнения. Является основой для тестовых утверждений (`Assert`).
- **Инвариант (Invariant)** — условие, сохраняющее истинность между вызовами методов (например, уникальность идентификаторов, целостность коллекции).

2.2. Автоматизация тест-дизайна

Формальное описание позволяет программно применять классические методы тест-дизайна:

- **Классы эквивалентности** — разбиение входного домена на группы с одинаковым поведением. Каждый класс покрывается минимум одним тестом.
- **Граничные значения** — проверка точек на границах классов (пустая строка, `null`, `0`, `int.MaxValue`, недопустимые ID).

- **Тестовый оракул** — механизм определения ожидаемого результата. При генерации оракул строится автоматически из полей `post`, `exceptions` и `expected` спецификации.

2.3. Интеграция в CI/CD (GitHub Actions)

GitHub Actions позволяет декларативно описывать автоматизированные конвейеры. Для генерации тестов критичны:

- **Workflow trigger** — запуск при `push`, `pull_request` или вручную (`workflow_dispatch`).
 - **Pipeline steps** — последовательность действий: `checkout` → `setup .NET` → `run generator` → `dotnet test` → `upload artifacts`.
 - **Deterministic generation** — генератор должен быть детерминированным: одинаковая спецификация → одинаковые тесты. Это гарантирует стабильность пайплайна.
-

3. Практическое задание и порядок выполнения

 **Организационная справка:** Индивидуальные задания в классическом понимании отсутствуют. Каждый студент работает в собственной репозитории, содержащей уникальный тестируемый модуль. Все шаги демонстрируются на эталонном примере `IToDoListManager`. При выполнении лабораторной работы вы применяете описанную методологию к **вашему** модулю, сохраняя структуру репозитория и подход к формализации.

3.1. Исходная структура репозитория

Ваш репозиторий уже содержит базовую структуру:

```

MyProject/
├── src/
│   └── Module.Core/      # Библиотека с реализацией модуля
├── tests/
│   └── Module.Tests/    # Проект с тестами (JUnit)
├── .github/
│   └── workflows/
│       ├── ci.yml       # Базовый пайплайн (push + manual)
│       └── compare.yml  # Пайплайн для сравнения всех версий
└── README.md

```

Задача: дополнить репозиторий директориями `spec/`, `generator/`, файлом `config.yaml` и обновить `ci.yml`.

3.2. Пошаговое выполнение (на примере `ITodoListManager`)

Шаг 0. Исходный интерфейс и требования (база для формализации)

Студентам предоставляется следующий контракт модуля:

```

namespace Lab.Interfaces;

public interface IToDoListManager
{
    void AddTask(string description);
    void RemoveTask(int id);
    void MarkTaskAsCompleted(int id);
    string[] GetTasks();
}

```

Функциональные требования:

1. Добавление задачи: принимает описание, создаёт задачу с уникальным ID и статусом «не выполнена», добавляет в список.
2. Удаление задачи: принимает ID. Если существует — удаляет. Если не найдена — состояние не меняется.
3. Отметить как выполненную: принимает ID. Если существует и «не выполнена» → меняет статус на «выполнена». Если уже выполнена или не найдена → состояние не меняется.

4. Получение списка: возвращает массив строк. Каждая строка содержит ID, описание и статус.

Нефункциональные требования:

- Идентификаторы задач уникальны в пределах одного списка.
- Статус задачи может быть только «выполнена» или «не выполнена».
- Операции не нарушают целостность данных (например, не допускают дублирования ID).

Шаг 1. Создание формальной спецификации с явной трассировкой требований

Создайте файл `spec/todolist.yaml`. Каждый элемент спецификации содержит явную ссылку на номер функционального/нефункционального требования, что обеспечивает сквозную трассировку.

```
# spec/todolist.yaml

module: ToDoListManager

# Трассировка нефункциональных требований к глобальным инвариантам

non_functional_requirements:
  - id: NFR-1
    desc: "Идентификаторы задач уникальны в пределах одного списка"
  - id: NFR-2
    desc: "Статус задачи: только 'Не выполнена' или 'Выполнена'"
  - id: NFR-3
    desc: "Операции не нарушают целостность данных"

methods:
  - name: AddTask
    # Явная привязка к требованиям
    req_functional: ["1"]
    req_non_functional: ["NFR-1", "NFR-2", "NFR-3"]
    signature: "void AddTask(string description)"
    pre: "description != null && !string.IsNullOrEmpty(description)"
    post: "Создана задача с новым уникальным ID, Status = 'Не выполнена', Count = old(Count) + 1"
    equivalence_classes:
```

- case: "Валидное описание"
 - inputs: ["Написать отчёт"]
 - expected: "Задача добавлена, статус 'Не выполнена'"
- case: "Пустая строка (нарушение pre)"
 - inputs: [""]
 - expected: "ArgumentException или игнорирование"
- case: "Null (нарушение pre)"
 - inputs: [null]
 - expected: "ArgumentNullException или игнорирование"

- name: RemoveTask

req_functional: ["2"]

req_non_functional: ["NFR-3"]

signature: "void RemoveTask(int id)"

pre: "true"

post: "id существует → Count = old(Count) - 1; id не существует → State unchanged"

equivalence_classes:

- case: "Существующий ID"
 - inputs: [1]
 - expected: "Задача удалена"
- case: "Несуществующий ID"
 - inputs: [999]
 - expected: "Состояние системы не изменилось"

- name: MarkTaskAsCompleted

req_functional: ["3"]

req_non_functional: ["NFR-2", "NFR-3"]

signature: "void MarkTaskAsCompleted(int id)"

pre: "true"

post: "id существует && Status == 'Не выполнена' → Status = 'Выполнена'; else → State unchanged"

equivalence_classes:

- case: "Незавершённая задача"
 - inputs: [1]
 - expected: "Статус изменён на 'Выполнена'"
- case: "Уже завершённая задача"
 - inputs: [2]
 - expected: "Состояние не изменилось"
- case: "Задача не найдена"
 - inputs: [999]
 - expected: "Состояние не изменилось"

- name: GetTasks

req_functional: ["4"]

req_non_functional: ["NFR-1", "NFR-2"]

signature: "string[] GetTasks()"

pre: "true"

post: "Возвращает массив строк формата '{id}. {description} [{status}]'"

equivalence_classes:

- case: "Пустой список"

```
inputs: []
expected: "Массив длины 0"
- case: "Список с задачами"
inputs: []
expected: "Массив с отформатированными строками"
```

Комментарий для студента:

Поля `req_functional` и `req_non_functional` обеспечивают явное соответствие между текстовыми требованиями и элементами спецификации. Это позволяет автоматически генерировать матрицу покрытия требований и упрощает аудит.

Шаг 2. Конфигурация проекта

Создайте файл `config.yaml` в корне репозитория. Он фиксирует параметры генерации и исключает хардкод путей в скриптах.

```
# config.yaml

# Конфигурация генератора тестов

target_language: csharp

test_framework: nunit

spec_path: spec/todolist.yaml

output_dir: tests/Module.Tests
```

Шаг 3. Разработка генератора тестов (C#)

Создайте файл `generator/gen_tests.py`. Скрипт полностью ориентирован на C# и NUnit, подробно прокомментирован.

```
#!/usr/bin/env python3

# generator/gen_tests.py

"""
Генератор параметризованных NUnit-тестов на основе YAML-спецификации.
Читает формальное описание, преобразует классы эквивалентности в [TestCase],
генерирует структуру Arrange-Act-Assert и сохраняет файл в указанный каталог.
Зависимости: pyyaml (pip install pyyaml)
```

```
"""
```

```
import yaml
```

```
import argparse
```

```
import os
```

```
from pathlib import Path
```

```
from typing import Dict, List, Any
```

```
# -----
```

```
# 1. ШАБЛОНЫ КОДА
```

```
# -----
```

```
# Используем f-strings для рендеринга. Двойные фигурные скобки {{ }}
```

```
# экранируют литералы в C# коде (например, string.Format).
```

```
TEST_FILE_TEMPLATE = """//
```

```
=====
```

```
// AUTO-GENERATED TESTS. DO NOT EDIT MANUALLY.
```

```
// Source: {spec_source}
```

```
// Generator: gen_tests.py v1.0
```

```
// =====
```

```
using System;
```

```
using System.Collections.Generic;
```

```
using NUnit.Framework;
```

```
using Lab.Interfaces;
```

```
namespace Module.Tests
```

```
{{
```

```
    [TestFixture]
```

```
    [Description("Автоматически сгенерированные тесты для {module_name}")]
```

```
    public class {module_name}Tests
```

```
    {{
```

```
        private I{module_name} _sut;
```

```
        [SetUp]
```

```
        public void Setup()
```

```
        {{
```

```
            // Инициализация тестируемой системы (SUT)
```

```
            _sut = new {module_name}();
```

```
        }}
```

```
        {test_methods}
```

```
    }}
```



```

}}
"""

TEST_METHOD_TEMPLATE = """
[Test]
[Description("Класс эквивалентности: {case_desc}")]
{test_cases}
public void Test_{method_name}_{case_name}()
{{
    // === Arrange ===
    // Предусловие (Precondition): {pre}
    // Ожидаемый результат: {expected}

    // === Act ===
    {act_code}

    // === Assert ===
    // Постусловие (Postcondition): {post}
    {assert_code}
}}
"""

```

```

TEST_CASE_TEMPLATE = "[TestCase({inputs})]"

```

```

# -----

```

```

# 2. ПАРСИНГ СПЕЦИФИКАЦИИ

```

```

# -----

```

```

def load_spec(spec_path: str) -> Dict[str, Any]:
    """Безопасная загрузка YAML-спецификации."""
    with open(spec_path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f)

```

```

# -----

```

```

# 3. ГЕНЕРАЦИЯ КОДА

```

```

# -----

```

```

def format_csharp_input(value: Any) -> str:
    """Преобразует значение из YAML в литерал C#."""
    if value is None:
        return "null"
    if isinstance(value, str):
        return f'"{value}"'
    if isinstance(value, bool):

```

```

        return "true" if value else "false"
    return str(value)

def generate_method_tests(method_data: Dict[str, Any]) -> List[str]:
    """Генерирует один метод теста с параметризацией для всех классов
    эквивалентности."""
    case_blocks = []
    for eq_class in method_data.get("equivalence_classes", []):
        # Формируем списки входных параметров для [TestCase]
        inputs_str = ", ".join(format_csharp_input(inp) for inp in eq_class["inputs"])

        # Формируем Act-код (вызов метода)
        method_name = method_data["name"]
        act_code = f"_sut.{method_name}({inputs_str});"
        if method_data["signature"].startswith("void"):
            act_code = f"_sut.{method_name}({inputs_str});"
        else:
            # Если метод возвращает значение, сохраняем его в переменную
            act_code = f"var result = _sut.{method_name}({inputs_str});"

        # Формируем Assert-заглушку (в реальном генераторе здесь парсится поле post)
        assert_code = "Assert.Pass(\"Generated assertion placeholder. Replace with actual
        logic based on postcondition.\");"

        # Собираем блок теста
        case_blocks.append(
            TEST_METHOD_TEMPLATE.format(
                case_desc=eq_class["case"],
                test_cases=TEST_CASE_TEMPLATE.format(inputs=inputs_str),
                method_name=method_name,
                case_name=eq_class["case"].replace(" ", "_").replace("(", "").replace(")", ""),
                pre=method_data["pre"],
                expected=eq_class["expected"],
                act_code=act_code,
                post=method_data["post"],
                assert_code=assert_code
            )
        )
    return case_blocks

def render_and_save(spec: Dict[str, Any], config: Dict[str, Any]) -> None:
    """Собирает полный файл тестов и сохраняет на диск."""
    module_name = spec["module"]
    test_methods = []

    for method in spec["methods"]:
        test_methods.extend(generate_method_tests(method))

```

```

# Склеиваем методы в один блок
tests_block = "\n".join(test_methods)

# Рендерим файл
file_content = TEST_FILE_TEMPLATE.format(
    spec_source=config.get("spec_path", "N/A"),
    module_name=module_name,
    test_methods=tests_block
)

# Сохраняем
out_dir = Path(config.get("output_dir", "tests/Module.Tests"))
out_dir.mkdir(parents=True, exist_ok=True)
output_file = out_dir / f"{module_name}Tests.Generated.cs"

output_file.write_text(file_content, encoding="utf-8")
print(f"[✓] Сгенерирован файл: {output_file}")
print(f"  Методов покрыто: {len(spec['methods'])}")
print(f"  Тестов сгенерировано: {sum(len(m.get('equivalence_classes', [])) for m in spec['methods'])}")

# -----

# 4. ТОЧКА ВХОДА

# -----

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="C# NUnit Test Generator from YAML Spec")
    parser.add_argument("--config", default="config.yaml", help="Путь к config.yaml")
    args = parser.parse_args()

    print("[*] Загрузка конфигурации...")
    with open(args.config, "r", encoding="utf-8") as f:
        config = yaml.safe_load(f)

    print(f"[*] Загрузка спецификации: {config['spec_path']}...")
    spec_data = load_spec(config["spec_path"])

    print("[*] Генерация C# тестов...")
    render_and_save(spec_data, config)
    print("[✓] Готово.")

```

Шаг 4. Настройка CI/CD пайплайна (GitHub Actions)

Обновите `.github/workflows/ci.yml`. Пайплайн автоматически запускает генератор, собирает проект и исполняет тесты.

```
# .github/workflows/ci.yml
```

```
name: CI - Generate & Run NUnit Tests
```

```
on:
```

```
  push:
```

```
    branches: [ main, develop ]
```

```
  pull_request:
```

```
  workflow_dispatch:
```

```
jobs:
```

```
  test-generation-and-execution:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      # 1. Получение кода
```

```
      - name: Checkout repository
```

```
        uses: actions/checkout@v4
```

```
      # 2. Настройка среды Python для генератора
```

```
      - name: Setup Python
```

```
        uses: actions/setup-python@v5
```

```
        with:
```

```
          python-version: '3.11'
```

```
      - name: Install dependencies
```

```
        run: pip install pyyaml
```

```
      # 3. Генерация тестов из спецификации
```

```
      - name: Generate NUnit tests
```

```
        run: python generator/gen_tests.py --config config.yaml
```

```
      # 4. Настройка .NET SDK
```

```
      - name: Setup .NET
```

```
        uses: actions/setup-dotnet@v4
```

```
        with:
```

```
          dotnet-version: '8.0.x'
```

```
      # 5. Восстановление пакетов и сборка
```

```
      - name: Restore & Build
```

```
        run: dotnet build MyProject.sln --configuration Release
```

```
      # 6. Запуск тестов (включая сгенерированные)
```

```
      - name: Run Tests
```

```
        run: dotnet test tests/Module.Tests/Module.Tests.csproj --logger "trx;LogFileName=test-results.trx" --verbosity normal
```

```
      # 7. Сохранение результатов в артефакты
```

```
      - name: Upload test results
```

```
        if: always()
```

```
uses: actions/upload-artifact@v4
with:
  name: nunit-test-results
  path: |
    tests/**/TestResults/
    tests/**/test-results.trx
```

Шаг 5. Локальная проверка и публикация

1. Убедитесь, что `src/Module.Core/` содержит реализацию `ITodoListManager`.
2. Запустите генератор локально: `python generator/gen_ tests.py`
3. Откройте `tests/Module.Tests/TodoListManagerTests.Generated.cs`, проверьте корректность синтаксиса.
4. Выполните `dotnet test` для локальной валидации.
5. Закоммитьте изменения и отправьте в репозиторий:

```
git add spec/ generator/ config.yaml .github/workflows/ci.yml
git commit -m "feat: add formal spec, C# test generator, CI pipeline"
git push origin main
```

6. В интерфейсе GitHub проверьте вкладку **Actions**: пайплайн должен завершиться статусом `success`. Скачайте артефакт `nunit-test-results` для анализа.

3.3. Требования к отчету

Отчёт формируется индивидуально на основе **вашего** репозитория и тестируемого модуля. Структура отчёта строго регламентирована:

1. **Титульный лист** (ФИО, группа, тема, дата).
2. **Цель работы** (согласно разделу 1).
3. **Исходные данные и формализация**
 - Текст интерфейса/модуля.
 - Функциональные и нефункциональные требования.

- Файл YAML-спецификации с выделением трассировки (`req_functional`, `req_non_functional`). Поясните, как каждый пункт требований отражён в `pre`, `post`, `equivalence_classes`.

4. Реализация генератора

- Листинг `gen_tests.py` (или ссылка на commit).
- Описание логики работы: парсинг YAML, маппинг в C#-литералы, рендеринг шаблонов, генерация `TestCase`.
- Пример сгенерированного NUnit-файла с комментариями к ключевым блокам (Arrange, Act, Assert).

5. Автоматизация в GitHub

- Скриншот структуры репозитория после дополнения.
- Листинг `.github/workflows/ci.yml` с пояснением шагов.
- Скриншоты успешного выполнения пайплайна, загрузки артефактов, отчёта NUnit.

6. Анализ результатов

- Оценка покрытия классов эквивалентности.
- Трассировка: таблица "Требование → YAML-блок → Сгенерированный тест".
- Оценка детерминированности генерации и воспроизводимости в CI.
- Выводы о применимости подхода к вашему модулю.

7. Приложения (ссылка на репозиторий, хеш последнего коммита, инструкция по запуску).

4. Список рекомендуемых источников

1. Myers G.J., Sandler C., Badgett T. *The Art of Software Testing*. 3rd ed. Wiley, 2011.
2. Meyer B. *Object-Oriented Software Construction*. 2nd ed. Prentice Hall, 1997.

(Раздел 11: Design by Contract)

3. Ammann P., Offutt J. *Introduction to Software Testing*. 2nd ed. Cambridge University Press, 2016.
 4. NUnit Documentation. *Test Attributes & Parameterized Tests*. URL: <https://docs.nunit.org/>
 5. Microsoft Learn. *GitHub Actions for .NET*. URL: <https://learn.microsoft.com/en-us/dotnet/devops/>
 6. ISO/IEC/IEEE 29119-3:2013 *Software testing — Part 3: Test documentation*.
 7. GitHub Docs. *Workflow syntax for GitHub Actions*. URL: <https://docs.github.com/en/actions/using-workflows>
-

5. Контрольные вопросы

1. В чём принципиальное отличие формальной спецификации от технического задания? Какие свойства YAML делают его пригодным для автоматической генерации тестов?
2. Как поля `pre`, `post` и `equivalence_classes` в спецификации транслируются в паттерн Arrange-Act-Assert NUnit? Приведите пример.
3. Почему генератор тестов должен быть детерминированным? Какие последствия для CI/CD возникают при нарушении этого требования?
4. Как в предложенной архитектуре обеспечивается сквозная трассировка требований? Почему это важно для аудита и сертификации ПО?
5. Какие риски возникают при автоматическом генерировании `Assert` на основе текстовых постулов? Как можно улучшить генератор для автоматического вывода корректных утверждений?
6. Как пайплайн GitHub Actions обрабатывает ситуации, когда генерация успешна, но скомпилированные тесты падают? На каком этапе происходит разделение ответственности?

7. Почему сгенерированные тесты помечены как `DO NOT EDIT`? Какие стратегии используются в индустрии для работы с автогенерируемым кодом в репозитории?
8. Как предложенный подход масштабируется на модули с асинхронными методами, зависимостями (DI) или взаимодействием с внешними системами (БД, HTTP)? Что потребуется добавить в спецификацию и генератор?
-